Fixing a PAM Detection Gap in Wazuh

Maryam Shahzad

wazuh/wazuh pull request #38270, targeting branch 4.14.9

July 2026

Status: submitted, awaiting maintainer review, merged after review.

Current status

Issue: wazuh/wazuh #16252

Pull request: wazuh/wazuh #38270

Target branch: 4.14.9

Pull request 38154: closed, superseded by 38270

Merge status: waiting on a maintainer review and on the repository's commit signature requirement, both standard for any outside contributor, neither a rejection

Why I picked this project: -

I wanted to contribute something real to an open source security project, not just a typo fix somewhere, but I also did not want to jump straight into the hardest possible thing and get stuck. Wazuh seemed like a good fit. It is a widely used SIEM and XDR platform, the codebase is large but the rule files are plain XML, so I could actually read them and understand what was happening without needing to compile C code first.

I went looking through open issues on wazuh/wazuh and found issue #16252, titled PAM frequency rule not generating alert. It had been open for a while with no fix. That felt like a good sign. It meant the bug was real and nobody had gotten to it yet, not that it was too hard for anyone to solve.

Understanding the bug: -

Rule 5551 in the PAM ruleset is supposed to catch repeated failed logins. It fires after eight failures inside a three minute window, but only if all of those failures share the same source IP address. That condition is called same source ip in the rule file.

I traced through how that source IP actually gets set. The PAM decoder pulls it from the rhost field in the raw log line. For something like SSH, rhost is filled in with the client address, so the rule works fine. But for local authentication attempts, things like su or login, there is no remote host at all. The log line just has rhost with nothing after it. A real example looks like this.

```
su(pam_unix)[23164]: authentication failure; logname= uid=1342 euid=0 tty= ruser=dcid
rhost= user=osaudit
```

Because rhost is empty, the decoder never extracts a source IP for these events. With no IP to compare across the eight failures, the same source ip condition can never be true, so rule 5551 stays silent no matter how many times someone hammers su on the same box. That is a real gap. Local brute forcing of su is a pretty standard thing to see after someone has already gotten a low privilege shell and is trying to escalate.

Designing the fix: -

Before writing anything, I searched the rest of the ruleset to see if this exact problem had come up before. It had. A few other rule files, the Windows authentication rules, the Serv U FTP rules, and the RSA Auth Manager rules, all have a sibling rule that correlates on same user instead of same source ip, specifically for services where an IP is not reliably available. So the pattern I needed already existed in the codebase, I was not inventing something new, I was just applying it to PAM.

The new rule I added is numbered 5558. It keeps the same frequency and timeframe as 5551, eight events in one hundred eighty seconds, but it correlates on the decoded user field instead of the source IP.

```
<rule id="5558" level="10" frequency="8" timeframe="180">
  <if_matched_sid>5503</if_matched_sid>
  <same_user />
  <description>PAM: Multiple failed logins for the same user in a small period of
  time.</description>
  <mitre>
    <id>T1110</id>
  </mitre> <group>authentication_failures,pci_dss_10.2.4,pci_dss_10.2.5,
  pci_dss_11.4,gpg13_7.8,gdpr_IV_35.7.d,gdpr_IV_32.2,
  hipaa_164.312.b,nist_800_53_AU.14,nist_800_53_AC.7,
  nist_800_53_SI.4,tsc_CC6.1,tsc_CC6.8,tsc_CC7.2,tsc_CC7.3,</group>
</rule>
```

Rule 5551 itself does not change at all. This is purely additive, so anything already relying on the original rule keeps working exactly as before.

Everything the final diff touches fits in one table.

Field	Value
File changed	`ruleset/rules/0085-pam_rules.xml`
Lines added	11
Existing rules modified	none
New rule	5558
Behavior added	detects repeated PAM authentication failures for the same user when source IP correlation isn't possible

Setting up the workspace: -

I forked wazuh/wazuh into my own GitHub account and cloned it onto my machine using Git Bash on Windows.

```
$ git clone https://github.com/MaryamShazz/wazuh.git
$ cd wazuh
```

The plan was to branch off the current stable release, v4.14.7, since that is the version where I confirmed the bug still exists. The first attempt at checking out that tag failed.

```
$ git checkout v4.14.7
error: pathspec 'v4.14.7' did not match any file(s) known to git
```

It took me a moment to realise why. When GitHub creates a fork it copies the main branch only, it does not bring along the tags from the original repository. My fork simply did not have that tag yet. Fetching tags from my own fork did not help either, since the tag was never pushed there in the first place.

```
$ git tag
$ git fetch --tags
$ git tag
```

Nothing came back. So I added the original repository as a second remote, called upstream, and fetched its tags directly.

```
$ git remote add upstream https://github.com/wazuh/wazuh.git
$ git fetch upstream --tags
```

That pulled down a huge number of branches and tags, thousands of them, going back to version 1.0. I filtered down to the ones I actually needed.

```
$ git tag | grep v4.14
v4.14.0 v4.14.1
v4.14.2 v4.14.3
v4.14.4 v4.14.5
v4.14.6 v4.14.7
```

With v4.14.7 now available locally, I checked it out and branched from it.

```
$ git checkout v4.14.7
$ git checkout -b fix/16252-pam-frequency-same-user
```

Applying the fix and a mistake I made: -

I had the rule change ready as a patch file, so I applied it with git am, which keeps the full commit message attached rather than just the code change.

```
$ git am 0001-Add-same_user-PAM-frequency-rule-to-catch-local-auth.patch Applying: Add
same_user PAM frequency rule to catch local auth-failure bursts without a source IP
```

The commit went through and git log showed exactly what I expected, the rule addition along with the explanation of rhost being empty for su and login. What I did not realise at the time was that I ran

that same git am command a second time by mistake, thinking the first one had not gone through because I did not see obvious confirmation output. It had gone through. Running it again applied the same patch on top of itself, which quietly duplicated the entire rule block inside the XML file. I did not catch this until later, during testing.

Getting a Wazuh manager running to actually test this: -

Reading the rule and convincing myself it was logically correct was not enough, I wanted to see it fire. I tried the official wazuh-docker repository first, both the current single node compose stack and an older branch of it, and both failed because the images they referenced no longer exist on Docker Hub, one pointed at a 5.1.0 indexer image that was never published, the other at a Kibana image that had been removed entirely.

I did not actually need the dashboard or the indexer to test one detection rule, wazuh-logtest only needs the manager itself, so I switched strategy and pulled the manager image directly.

```
$ docker pull wazuh/wazuh-manager:4.14.6
$ docker run -d --name wazuh-test wazuh/wazuh-manager:4.14.6
```

That worked right away and gave me a running container I could work with.

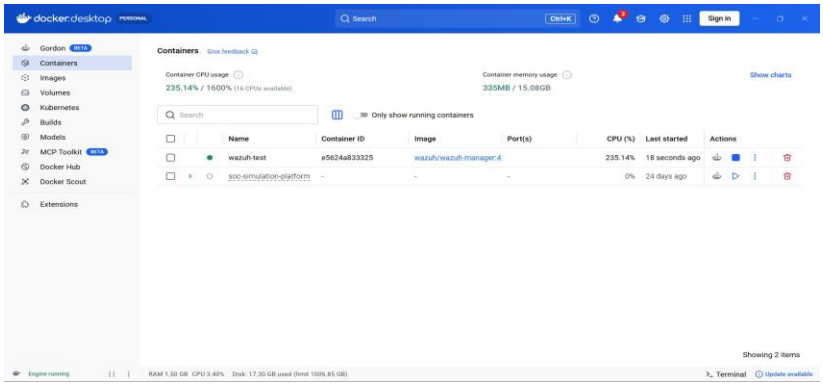


Figure 1: Docker Desktop showing the wazuh-test container running from wazuh/wazuhmanager:4.14.6.

Finding and fixing the duplicate rule: -

I copied my locally modified rule file straight into the running container so I could test without needing to commit anything yet.

```
$ docker cp ruleset/rules/0085-pam_rules.xml wazuh-test:/var/ossec/ruleset/rules/0085pam_rules.xml
```

When I restarted the Wazuh services inside the container, a warning appeared that I had not seen before.

```
WARNING: (7612): Rule ID '5558' is duplicated. Only the first occurrence will be considered.
```

That is when I realised what had happened with the double git am earlier. I grepped the file to confirm it.

```
$ grep -n 'id="5558"' ruleset/rules/0085-pam_rules.xml
81: <rule id="5558" level="10" frequency="8" timeframe="180">
91: <rule id="5558" level="10" frequency="8" timeframe="180">
```

Two identical copies of the same rule, one right after the other. I opened the file in an editor, deleted the second copy, saved it, and copied the corrected version back into the container.

```
$ code 0085-pam_rules.xml
$ docker cp 0085-pam_rules.xml wazuh-test:/var/ossec/ruleset/rules/0085-pam_rules.xml
```

Then I went back to the actual git repository, staged the corrected file, and committed the cleanup as its own commit, separate from the original fix, so the history honestly reflects that a mistake happened and was corrected.

```
$ git add 0085-pam_rules.xml
$ git commit -m "Remove duplicate rule 5558"
$ git push
```

Later, once the branch had been rebuilt on 4.14.9, I ran the same uniqueness check again against the manager, alongside a status check confirming every Wazuh service was actually running.

Watching it actually fire: -

With the corrected file in place, I restarted the manager again, this time with no duplicate warning, and ran wazuh-logtest, feeding it the same su authentication failure line repeatedly.

```
Jul 31 20:35:00 host su: pam_unix(su:auth): authentication failure; logname= uid=1000 euid=0
tty=/dev/pts/0 ruser=alice rhost= user=root
```

The first seven times I submitted that line, it matched only the base rule, 5503, the ordinary single failed login rule. The firedtimes counter climbed from one up to seven, and nothing else happened. Then on the eighth submission, inside the same one hundred eighty second window, rule 5558 fired.

That was the moment I actually believed the fix worked, not just in theory but against a real running manager. Repeated local PAM failures with no source IP at all now generate an alert, exactly the case that rule 5551 was blind to.

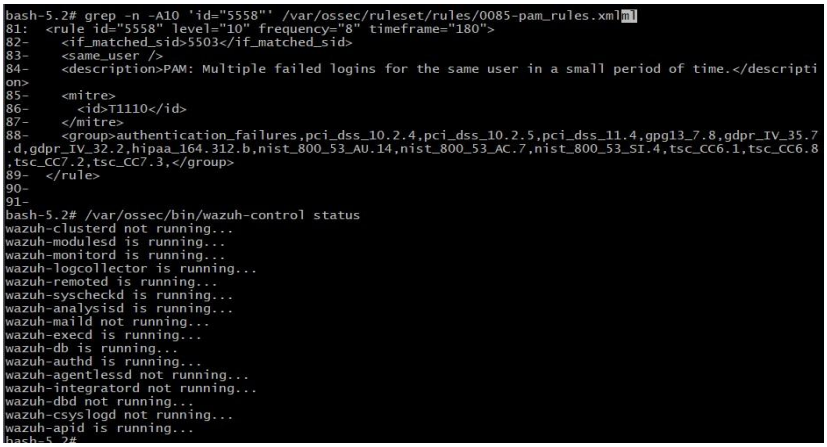


Figure 2: Confirming rule 5558 appears exactly once in the deployed ruleset, and that all Wazuh services are running, before relying on any test result against this container.

Opening the pull request: -

I pushed the branch and opened a pull request from my fork into wazuh:main, referencing the issue directly so GitHub would link the two automatically.

```
$ git push -u origin fix/16252-pam-frequency-same-user
```

The description explains the root cause, the fix, and includes the wazuh-logtest output above as evidence. The pull request is number 38154. I later found out this was opened against the wrong branch, which the next section covers, but at the time it felt like the finish line.

Finding out the pull request was aimed at the wrong branch: -

A few days after opening the pull request I went back to double check the diff, comparing my branch against the actual upstream repository instead of just my own fork.

```
$ git remote add upstream https://github.com/wazuh/wazuh.git
$ git fetch upstream main
$ git diff upstream/main...HEAD -- ruleset/rules/0085-pam_rules.xml
```

That diff looked fine on its own, but out of caution I tried rebasing my branch onto the latest upstream main just to see if anything had changed underneath me.

```
$ git rebase upstream/main
CONFLICT (modify/delete): ruleset/rules/0085-pam_rules.xml deleted in HEAD and modified in 2f15f86be7
(Add same_user PAM frequency rule to catch local auth-failure bursts without a source IP). Version
2f15f86be7 left in tree.
```

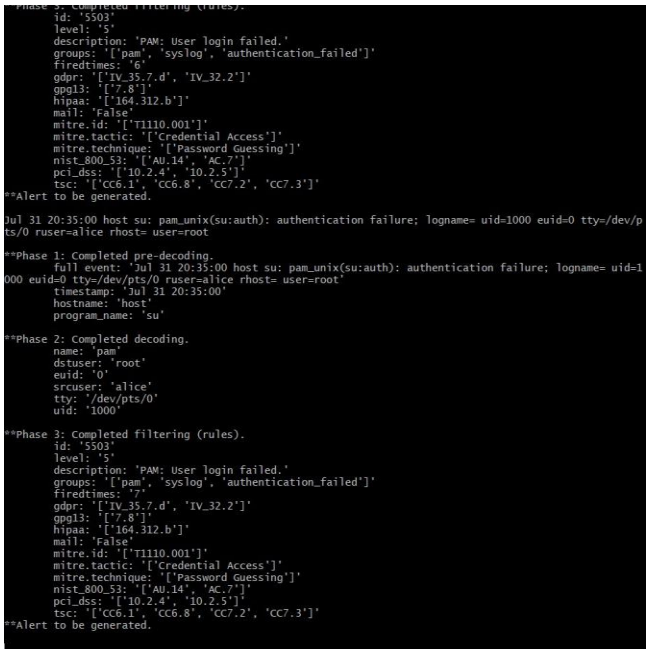


Figure 3: PAM authentication failure at the seventh event. The repeated local authentication failure continues to match rule 5503, with firedtimes: ’7’. Rule 5558 has not yet triggered because its configured frequency threshold of eight events has not been reached.

A modify or delete conflict means one side changed a file and the other side deleted it entirely. I checked which side had actually deleted it.

```
$ git show upstream/main:ruleset/rules/0085-pam_rules.xml fatal: path
’ruleset/rules/0085-pam_rules.xml’ exists on disk, but not in ’upstream/main’
```

The file simply is not on main anymore. Wazuh has been moving the classic XML ruleset out of that branch as part of a larger rework of their detection engine for version five and six, and main had already moved past the point where PAM rules live in that folder. My pull request, opened against main, was asking to add eleven lines to a file that branch does not even have. It would never merge as it stood, no matter how correct the fix itself was.

I abandoned that rebase and looked for the right branch instead.

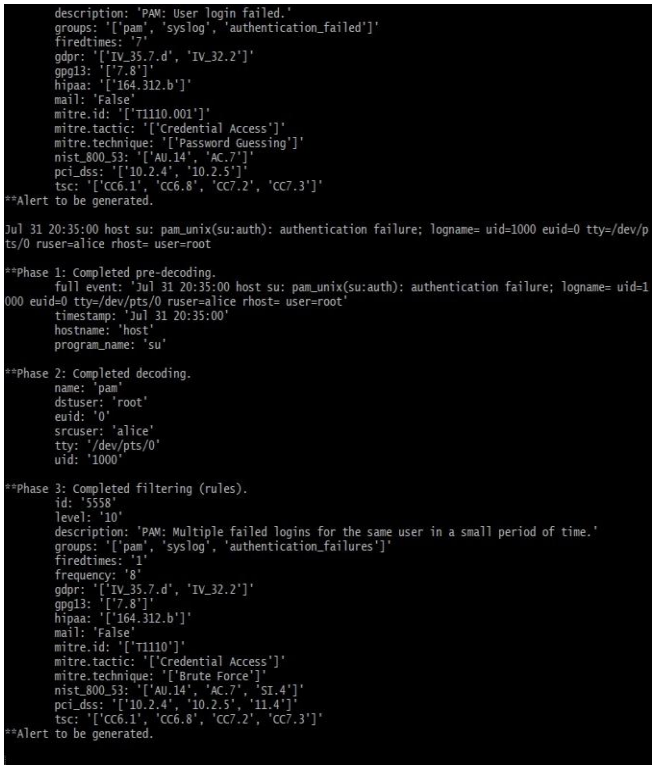


Figure 4: Wazuh-logtest validation of rule 5558. The eighth repeated local PAM authentication failure triggers rule 5558 with frequency 8, level 10, and MITRE ATT&CK technique T1110 (Brute Force), demonstrating detection of repeated local authentication failures without a source IP.

```
$ git rebase --abort
$ git ls-remote --heads upstream "4.*"
```

That listed the maintenance branches, including one called 4.14.9, newer than the 4.14.7 tag I had originally branched from. I fetched it and checked whether the PAM rules still lived where I expected.

```
$ git fetch upstream 4.14.9
$ git show upstream/4.14.9:ruleset/rules/0085-pam_rules.xml | grep -n -E 'id="5503"|id
    ="5551"|id="5557"'
33: <rule id="5503" level="5">
71: <rule id="5551" level="10" frequency="8" timeframe="180">
112: <rule id="5557" level="5">
```

Good, the file and the rules I was building on top of were both still there. That was the branch

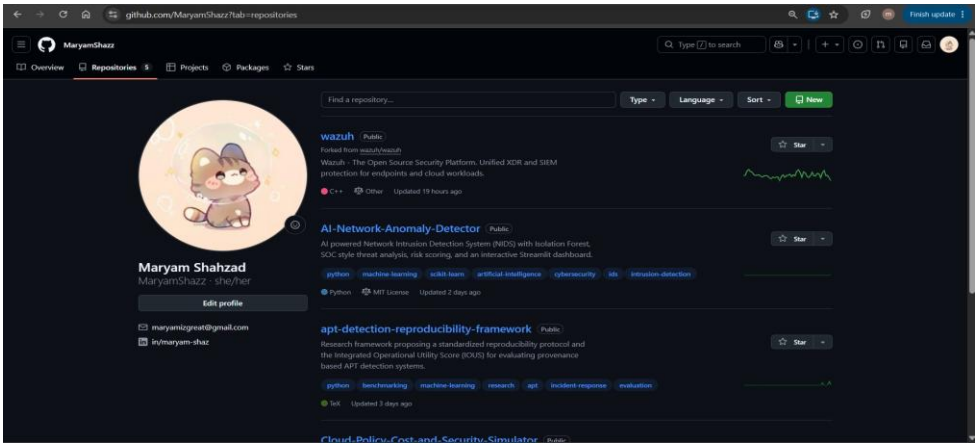


Figure 5: My GitHub profile showing the forked wazuh repository under my account.

my fix actually belonged on.

Rebuilding the branch cleanly on 4.14.9: -

I rebased onto that branch instead of main.

```
$ git rebase upstream/4.14.9
Successfully rebased and updated refs/heads/fix/16252-pam-frequency-same-user.
```

That preserved my three commits, including the duplicate rule mistake and its cleanup, just replayed on top of 4.14.9 instead of the old 4.14.7 tag. I wanted the final history to be simpler than that though, three commits where one of them exists purely to fix a mistake I made in an earlier commit is not something I wanted sitting in front of a maintainer. I reset the branch back to where it forked from upstream and made a single clean commit instead.

```
$ git reset --soft upstream/4.14.9
```

```
$ git commit -m "Add same_user PAM frequency rule"
```

Before pushing anything I checked the result properly. I confirmed the diff only touched the one file I intended.

```
$ git diff upstream/4.14.9...HEAD --stat ruleset/rules/0085-
 pam_rules.xml | 11 ++++++++++ 1 file changed, 11
 insertions(+)
```

I confirmed the XML was still valid.

```
$ python -c "import xml.etree.ElementTree as ET; ET.parse('ruleset/rules/0085-pam_rules.
      xml'); print('XML OK')"
XML OK
```

I confirmed rule 5558 appeared exactly once, not duplicated again.

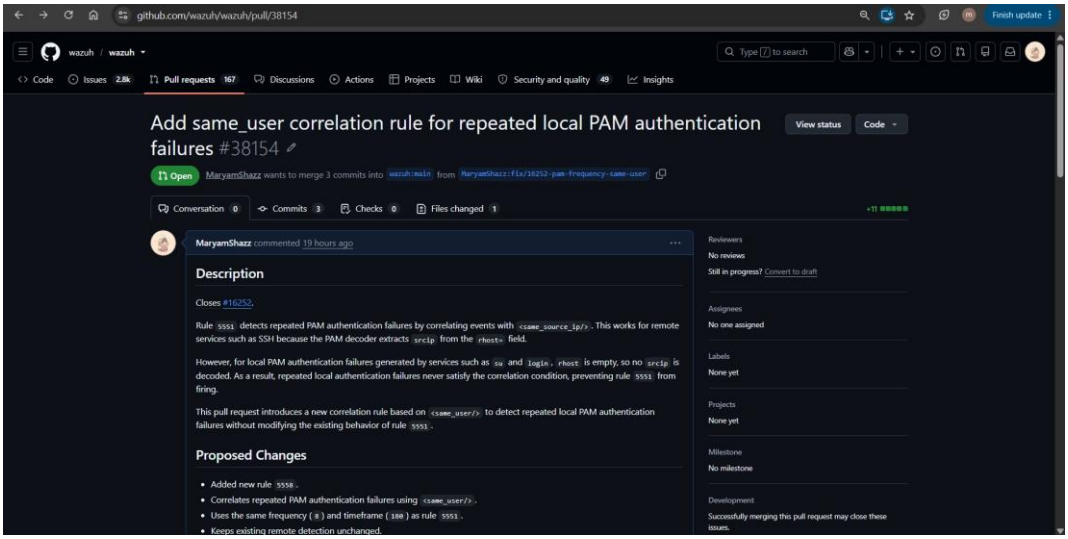


Figure 6: Pull request 38154 as it looked when first opened against wazuh:main, before I later closed it in favor of 38270.

```
$ grep -n 'id="5558"' ruleset/rules/0085-pam_rules.xml 81: <rule id="5558"
level="10" frequency="8" timeframe="180">
```

And I ran a whitespace and formatting check on the diff itself.

```
$ git diff upstream/4.14.9...HEAD --check
```

That command produced no output at all, which is what you want to see, it means there were no trailing whitespace problems or other formatting issues for git to complain about. With all of that checked, I force pushed the cleaned up branch. I used force with lease rather than a plain force push, since that version refuses to overwrite the branch if someone or something else had changed it on the remote since I last looked, which felt like the safer habit to build even though I was the only one touching this branch.

```
$ git push --force-with-lease origin fix/16252-pam-frequency-same-user
+ 5d12fbab14...81c1c61253 fix/16252-pam-frequency-same-user -> fix/16252-pam-frequencysame-user (forced update)
```

A second pull request, this time against the branch that can actually take it:-

With the branch cleaned up and sitting on the correct base, I opened a new pull request, this time from that same branch into wazuh:4.14.9 instead of wazuh:main. It is pull request number 38270, a single commit, an eleven line diff, nothing else attached to it.

I want to be honest about something here rather than gloss over it. Because both pull requests point at the same branch name in my fork, opening the second one did not automatically clean up the first. Pull request 38154, the original one against main, was technically still sitting open, and since main does not contain the file at all, it was never going to be mergeable as it stood. So I went back and closed it myself, rather than leaving a maintainer to work out why one of my two open pull requests could never merge. Two open pull requests for the same issue is not a good look, and closing the one that could not go anywhere is part of finishing this properly, not an optional extra step.

Closing 38154 and what the merge status on 38270 actually means:-

I went back to pull request 38154 and closed it myself, with a comment explaining exactly why rather than just closing it silently.

```
This PR targets main, where the legacy ruleset/rules/0085-pam_rules.xml file is no longer present. I created PR #38270 targeting 4.14.9, where this ruleset is still applicable. Closing this PR in favor of #38270.
```

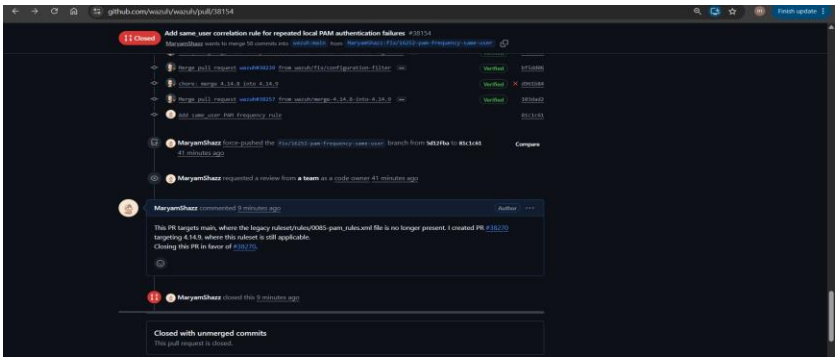


Figure 7: Pull request 38154 closed, with the comment explaining the reason and pointing to 38270.

One odd thing shows up on that closed page that is worth explaining rather than hiding. Right before I closed it, GitHub was listing 38154 as wanting to merge fifty eight commits into main, up from the original three. That looks alarming at first glance, as if the change had somehow grown enormous, but it has a boring explanation. Once I rebased my branch onto upstream 4.14.9 and force pushed, GitHub recalculated the diff between that branch and main from scratch. Since 4.14.9 and main have diverged a fair amount, with main having moved on to the newer engine structure, the comparison between my branch and main now walked through every commit that separates those two lines of history, not just the one I had written. It was never actually fifty eight commits of my own work, it was a side effect of comparing two branches that had grown apart, and it is exactly the kind of confusing artifact that closing 38154 avoided leaving for a maintainer to puzzle over.

Pull request 38270 is the one still open, and it currently shows as blocked from merging.

The reasons listed are that commits must have verified signatures, that at least one approving review from someone with write access is required, and that I am not authorized to push directly

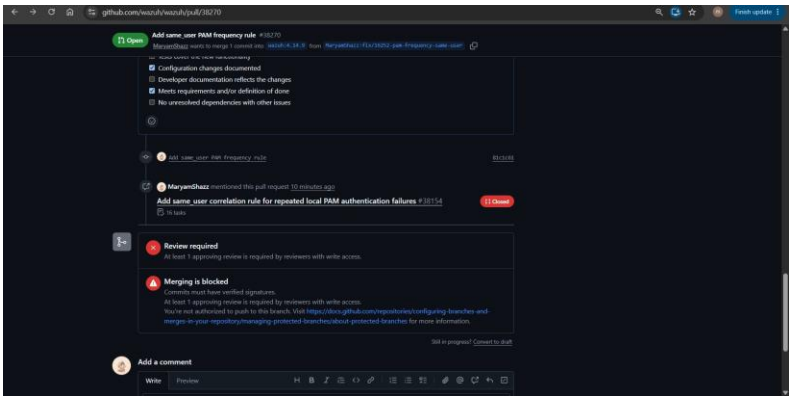


Figure 8: Pull request 38270, showing the standard review and signature requirements before it can merge.

to that branch myself. None of that is a rejection or a problem with the fix. It is just wazuh's repository protection rules applying to every external contributor by default, the same way they would to anyone else's pull request. The verified signature requirement means I would need to set up commit signing with a GPG or SSH key for that commit to satisfy it, which I have not done yet, and the review requirement simply means a maintainer with write access has to actually look at it before it can go in. Neither of those is something I can push through myself, they are the normal gate every outside contribution passes through.

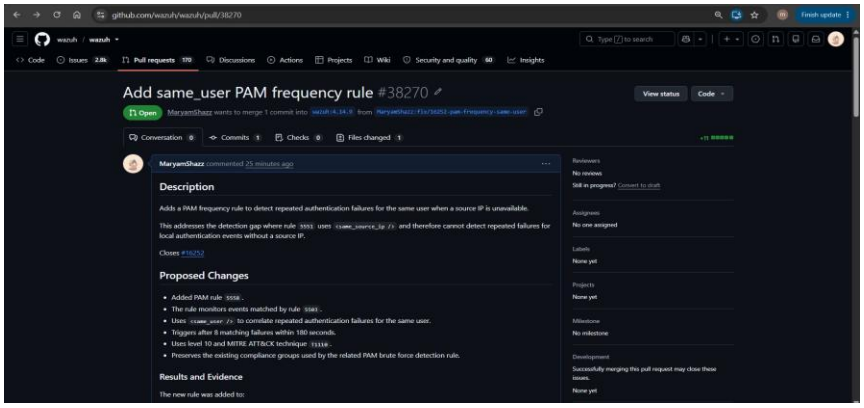


Figure 9: Pull request 38270 as it stands now, open against wazuh:4.14.9 with a clean single commit diff.

Evidence summary:-

Check	Result
XML parsing	pass
Rule 5558 uniqueness	pass
git diff --check	pass
Wazuh manager startup	pass
Local PAM failure with no source IP	reproduced
Seven repeated failures	matched rule 5503
Eighth repeated failure	matched rule 5558
MITRE mapping	T1110

Pull request opened	#38270
Upstream merge	pending

What I took away from this: -

The technical part, writing an XML rule that keys on a different field, was honestly the easy part once I understood the decoder. Most of what actually took time had nothing to do with detection engineering as such.

1. Validate patch application. Running git am twice can silently duplicate a change, and the tool will not warn you about it, only the running system will.
2. Compare against upstream, not only your own fork. A fork can look up to date and still be missing tags or sitting on a branch that has quietly fallen out of sync with where the real project has moved.
3. Check the target branch before opening a pull request, not after. The fix itself was correct from the start, but main was no longer where that file lived, and I only found that out by rebasing and watching it fail.
4. Validate behaviour, not just syntax. XML parsing proved the file was well formed, it did not prove the rule actually did anything. Only wazuh-logtest, run against a real manager, proved that.
5. Keep the final contribution minimal. The version that actually went up as pull request 38270 is one commit, one file, eleven lines added, nothing else attached to it.

If I am honest, the duplicate rule accident and the branch mismatch are probably the two most useful parts of this whole report to look back on later. Neither one is a clever technical insight, they are both just reminders that the tool will tell you when something is wrong if you are paying attention, and that fixing your own mistake openly, closing a pull request that could not merge instead of leaving it open and hoping nobody notices, is part of the work and not a failure of it.

As of now, pull request 38154 is closed and pull request 38270 is the only one still open, sitting against the branch where the fix can actually be merged, currently waiting on commit signing and a maintainer review before it can go any further. I am treating this as a first contribution rather than a finished one. The plan is to set up commit signing, keep an eye on the 38270 thread, respond to whatever a maintainer asks for, and use whatever comes back as the next thing to learn from.
